



INFORME DE LA PRÁCTICA EVALUABLE: “MORSE”

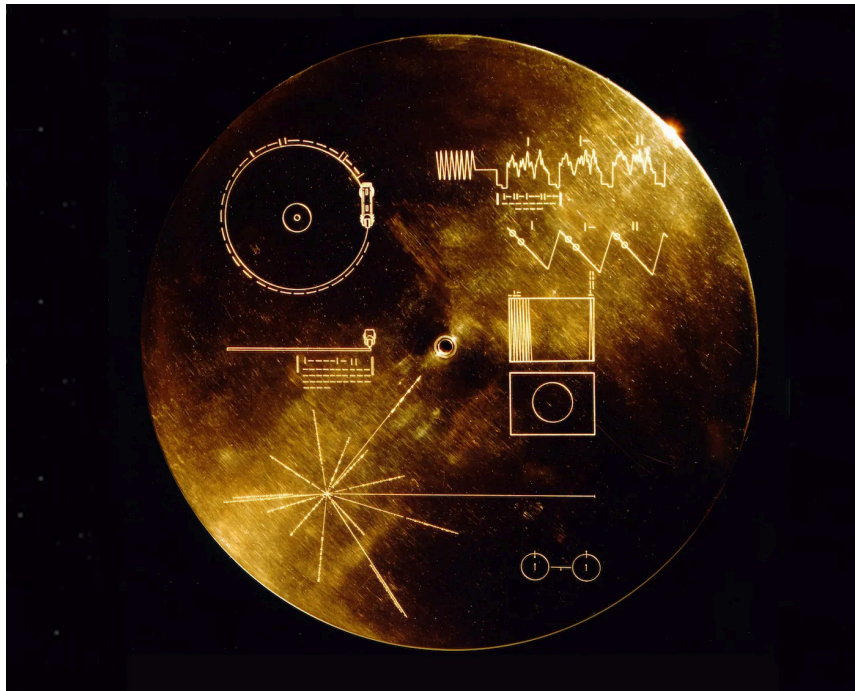


Imagen de la portada: disco de oro de la misión Voyager 1 [\[1\]](#).

BLANCO DE LA IGLESIA, DAVID.

RODRÍGUEZ FERNÁNDEZ, NICOLÁS.

1. Índice interactivo

1. Índice interactivo.....	2
2. Objetivo y aspectos relevantes.....	2
3. Diseño de la correspondencia entre códigos.....	2
4. Estructura del proyecto: análisis de su arquitectura.....	6
5. Archivo a archivo: descripción de funcionalidades.....	7
6. El código de <i>morse.asm</i>	10
7. Ejemplos de pruebas.....	12
8. Conclusiones, referencias y bibliografía.....	24
Anexo I. Historial de <i>commits</i> de GitHub.....	26

2. Objetivo y aspectos relevantes

El objetivo de esta práctica es diseñar y programar en lenguaje ensamblador ASM6809 un traductor bilateral entre Morse y texto. El diseño se explica detalladamente en los apartados desde el 2 hasta el 6, y la programación en ensamblador y resultados en terminal se detalla tanto en los apartados 6 y 7 como en el código fuente. Todos los archivos de código quedan adjuntos en el archivo .zip llamado «*PRACTICA_71080713H_45134230L*».

Las herramientas usadas durante todo el proceso de la práctica fueron: la máquina virtual de Raúl Alves en Oracle Virtualbox, Documentos de Google, VSCode, GitHub y el subsistema de Windows para Linux (WSL). La defensa presencial de esta práctica será realizada por sus dos autores ante el profesor de prácticas correspondiente, D. SERGIO BRAVO MARTÍN, el día miércoles, 13 de mayo de 2026, a las 18:00 horas.

3. Diseño de la correspondencia entre códigos

El estándar usado

Como se indica en el enunciado de la práctica, las correspondencias entre Morse y texto se usan para los números del 0 al 9 y para las letras de la A a la Z (independientemente de si estas últimas son mayúsculas o minúsculas), por lo que el traductor trabaja en el siguiente rango:

- Para el código Morse, se utiliza el estándar usado por la Unión Internacional de Telecomunicaciones, el ITU-R M.1677-1
- Para el texto, se utilizan los caracteres latinos alfanuméricos de estándar ASCII, con códigos 0030-0039, 0041-005A y 0061-007A.

El archivo *charMorse.asm*

Dentro de este archivo encontramos dos funciones principales para traducir:

- *char_a_morse*. Esta subrutina recibe un carácter ASCII y busca su equivalencia Morse recorriendo secuencialmente la tabla de caracteres. Cuando encuentra coincidencia, devuelve la dirección de la cadena Morse correspondiente. Su funcionamiento es el siguiente:
 1. Se recibe un carácter en un registro del procesador.
 2. Se convierte a mayúscula si pertenece al rango de letras minúsculas ASCII.
 3. Se recorre la tabla *chars_table* comparando el carácter introducido.
 4. Cuando existe coincidencia, se calcula la posición equivalente en *morse_table.asm*.
 5. Se devuelve un puntero al código Morse asociado.
 6. Si no existe coincidencia, se devuelve el código de error.

Gracias a este diseño, la misma función sirve tanto para letras mayúsculas como minúsculas, reutilizando la misma tabla de búsqueda.

- *morse_a_char*. Esta función realiza el proceso inverso: recibe una cadena Morse compuesta por puntos y guiones y devuelve el carácter ASCII correspondiente. Su funcionamiento es similar al anterior:
 1. Se recibe un puntero a una cadena Morse terminada en \0.
 2. Se recorren secuencialmente todas las entradas de *morse_table.asm*.
 3. Cada entrada se compara mediante una rutina auxiliar de comparación de cadenas (*strcmp*).
 4. Cuando se detecta coincidencia, se obtiene el índice correspondiente.
 5. Se devuelve el carácter ASCII almacenado en *chars_table.asm*.
 6. Si no existe coincidencia, se devuelve el código de error.

El archivo *morse_table.asm*

El archivo *morse_table.asm* contiene las estructuras de datos necesarias para almacenar todas las equivalencias Morse utilizadas por el traductor. Su principal función es centralizar las correspondencias para que puedan ser utilizadas desde cualquier módulo del proyecto.

Dentro de este archivo se encuentran:

- La tabla de códigos Morse (*morse_table.asm*).
- Los caracteres ASCII asociados (*chars_table.asm*).
- Las definiciones exportadas mediante *.globl* para permitir el acceso desde otros módulos.

Cada entrada de la tabla *morse_table.asm* se almacena como una cadena terminada en \0, lo que facilita las operaciones de comparación y recorrido mediante subrutinas de manejo de cadenas. El archivo, tal y como se nos adjuntó en la práctica, es el siguiente:

```

1      .module tabla_morse
2
3      .include "morse_table.inc"
4
5  tabla_morse:
6      .asciz ".-   " ; A
7      .asciz "- ... " ; B
8      .asciz "-.-. " ; C
9      .asciz "-..  " ; D
10     .asciz ".     " ; E
11     .asciz "..-.  " ; F
12     .asciz "--.   " ; G
13     .asciz "....  " ; H
14     .asciz "..    " ; I
15     .asciz ".---  " ; J
16     .asciz "-.-   " ; K
17     .asciz ".- ..  " ; L
18     .asciz "--    " ; M
19     .asciz "-.    " ; N
20     .asciz "----  " ; O
21     .asciz ".--.  " ; P
22     .asciz "--.-  " ; Q
23     .asciz ".-..  " ; R
24     .asciz "...   " ; S
25     .asciz "-     " ; T
26     .asciz "..-   " ; U
27     .asciz "...-  " ; V
28     .asciz ".--   " ; W
29     .asciz "-..-  " ; X
30     .asciz "-.-.  " ; Y
31     .asciz "--..  " ; Z
32     .asciz "-----" ; 0
33     .asciz ".-----" ; 1
34     .asciz ".. ----" ; 2
35     .asciz "... --"  ; 3
36     .asciz "....-"  ; 4
37     .asciz "....."  ; 5
38     .asciz "-....."  ; 6
39     .asciz "-- ... " ; 7
40     .asciz "--- .. " ; 8
41     .asciz "-----" ; 9
42
43  tabla_morse_Char:      .byte      26
44  tabla_morse_Number:    .byte      10
45  tabla_morse_Total:     .byte      36
46
47      ;; SEPARACION ENTRE LETRAS : 1 espacio
48      ;; SEPARACION ENTRE PALABRAS: 2 espacios
49

```

Convenientemente se ha creado otro archivo llamado *chars_table.asm*, en el cual la posición de cada traducción se encuentra sincronizada con su carácter Morse:

```
1  .module chars_table
2
3  .include "chars_table.inc"
4
5  chars_table:
6      .ascii "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
7
8  chars_table_Char:      .byte      26
9  chars_table_Number:    .byte      10
10 chars_table_Total:     .byte      36
```

Así, la A se encuentra en la posición 0 de la *chars_table.asm* pero su traducción a Morse se encuentra en la posición 0 de la *morse_table.asm*. De este modo, podemos obtener, multiplicando la posición del carácter en la *chars_table.asm* por 6, y sumando eso a la dirección de inicio de *morse_table.asm*, la dirección de memoria de su traducción de una forma comprensible e intuitiva para el lector.

Al separar estas tablas de datos del resto de subrutinas, se consigue una estructura modular donde futuras modificaciones del estándar Morse podrían realizarse sin alterar el funcionamiento interno de las funciones de traducción.

4. Estructura del proyecto: análisis de su arquitectura

Nuestro proyecto está alojado en la plataforma de proyectos de programación llamada “GitHub”. El enlace del repositorio asociado se encuentra en la referencia [\[2\]](#). Esta es la forma en la que se ha estructurado el proyecto:

COMPII_PRACTICAFINAL

- ↳ **libraries**
 - ↳ charMorse.asm
 - ↳ stdio.asm
 - ↳ stringH.asm
- ↳ **submenu1**
 - ↳ mostrar_tabla.asm
- ↳ **submenu2**
 - ↳ caracter_a_caracter.asm
 - ↳ linea_a_linea.asm
 - ↳ palabra_a_palabra.asm
- ↳ **submenu3**
 - ↳ caracter_a_caracter.asm
 - ↳ linea_a_linea.asm
 - ↳ palabra_a_palabra.asm
- ↳ **tables**
 - ↳ chars_table.asm
 - ↳ chars_table.inc
 - ↳ morse_table.asm
 - ↳ morse_table.inc
- ↳ ensamblar.sh
- ↳ morse.asm
- ↳ readme.md

Leyenda: Carpetas en negrita, archivos en redonda.

La progresión del trabajo realizado se puede observar en el **ANEXO I** de este informe.

5. Archivo a archivo: descripción de funcionalidades

A continuación se describen los principales archivos que componen el proyecto y la funcionalidad específica de cada uno de ellos. Los archivos repetidos solo se describen una vez.

Archivo principal: *morse.asm*

Es el núcleo principal del programa. Contiene el menú principal, los submenús de opciones, la gestión de entrada desde teclado, el control general del flujo del programa y las llamadas a todas las subrutinas auxiliares. También se encarga de detectar opciones incorrectas y de finalizar correctamente la ejecución.

Carpeta *libraries*

charMorse.asm

Es la biblioteca encargada de las traducciones entre caracteres ASCII y códigos Morse. Incluye principalmente las funciones *char_a_morse* y *morse_a_char*. Contiene las rutinas auxiliares de búsqueda y validación y constituye el núcleo lógico del traductor.

stdio.asm

Es la biblioteca de entrada y salida. Contiene funciones reutilizables para mostrar cadenas por pantalla (*printStr*), mostrar saltos de línea (*printBreak*), leer cadenas desde teclado (*inputStr*), e imprimir números enteros y hexadecimales (funciones no usadas en el proyecto final, pero útiles a la hora de debuggear el código).

Estas rutinas abstraen el acceso a los puertos de E/S de la máquina virtual.

stringH.asm

Es la biblioteca de manipulación de cadenas. Esta incluye funciones auxiliares como *strcmp*, para comparar cadenas. Asimismo, en subrutinas recorre cadenas terminadas en “\0” y realiza validaciones de longitud. Estas funciones son esenciales para las búsquedas dentro de *tabla_morse*.

Carpeta *tables*

chars_table.asm

Este archivo define la tabla principal de caracteres ASCII soportados por el traductor. Incluye las letras de la A a la Z, los números del 0 al 9 y la exportación global de la tabla.

morse_table.asm

Este archivo contiene todas las cadenas Morse almacenadas en memoria y agrupadas de seis en seis. Es la base de datos principal del traductor.

chars_table.inc

Este es un archivo de inclusión encargado de declarar externamente los símbolos relacionados con la tabla de caracteres ASCII (*chars_table.asm*).

morse_table.inc

Este es un archivo de inclusión utilizado para declarar externamente la tabla de códigos Morse (*morse_table.asm*).

Carpeta submenu1

mostrar_tabla.asm

Este archivo implementa la opción «*Ver tabla*» del menú principal. Este sirve para recorrer las tablas de *chars_table* y *morse_table*. También muestra por pantalla todas las equivalencias y formatea correctamente la salida para facilitar la lectura.

Carpeta submenu2

Contiene todas las funciones relacionadas con la traducción de texto ASCII a Morse.

caracter_a_caracter.asm

Este archivo permite introducir un único carácter y obtener inmediatamente su traducción Morse. Su funcionamiento es así: lee de teclado, valida el carácter, hace una traducción individual y después gestiona los errores.

palabra_a_palabra.asm

Este archivo gestiona la traducción de palabras completas. Su funcionamiento es así: traduce secuencialmente carácter a carácter, separa cada una de las letras mediante espacios, finaliza la traducción mediante la tecla enter y valida su longitud.

linea_a_linea.asm

Este archivo implementa la traducción de líneas completas. Como características tiene: permite traducir automáticamente hasta 100 caracteres, controla los espacios entre palabras y evita desbordamientos de memoria.

Carpeta *submenu3*

Contiene las funciones encargadas de traducir de Morse a texto ASCII.

caracter_a_caracter.asm

Este archivo traduce códigos Morse individuales. Funciona de la siguiente manera: valida los símbolos “.” y “-”, comprueba la longitud máxima, hace una conversión inmediata a ASCII, y hace una gestión de los códigos inválidos.

palabra_a_palabra.asm

Este archivo permite introducir varias letras Morse consecutivas para formar palabras. Se usan espacios como separadores, se detectan los doble espacio como final de palabra, traduce automáticamente cada código válido y también gestiona errores parciales sin detener completamente la ejecución.

linea_a_linea.asm

Este archivo implementa la traducción de líneas completas en Morse. Presenta las siguientes funciones: hace una traducción continua de múltiples palabras, donde las palabras se separan mediante doble espacio, finalización con la tecla enter, detecta los errores y arroja los mensajes de error relacionados. Además, realiza una gestión eficiente de memoria reservada.

Otros archivos

ensamblar.sh

Se ha incluido en el proyecto un *script* de automatización encargado del ensamblado del proyecto. Este compila automáticamente todos los módulos ASM. Por lo tanto, este archivo ha facilitado las pruebas del proyecto durante su realización progresiva.

Los comandos "ensambla" y "limpia" son los utilizados durante las clases de prácticas de Computadores II. Se pueden instalar desde el studium de la asignatura.

El proyecto se ensambla ejecutando en terminal los siguientes comandos:

```
chmod +x ejecutar.sh  
./ensamblar.sh
```

readme.md

Este es el documento descriptivo del repositorio GitHub. Incluye el nombre de los autores, una explicación general del proyecto, las instrucciones de ejecución y un historial de versiones.

6. El código de morse.asm

A continuación se reproduce en dos columnas separadas el código fuente en ensamblador de morse.asm, archivo principal del proyecto:

```
.module morse

; MÓDULO PRINCIPAL: morse.asm
; Dependencias: stdio.asm, submenu1/mostrar_tabla.asm
; submenu2/caracter_a_caracter.asm,
; submenu2/palabra_a_palabra.asm
; submenu2/linea_a_linea.asm
; submenu3/caracter_a_caracter.asm,
; submenu3/palabra_a_palabra.asm,
; submenu3/linea_a_linea.asm

pantalla .equ 0xFF00
fin .equ 0xFF01
teclado .equ 0xFF02

.globl nucleo ; morse.asm

.globl printStr ; stdio.asm
.globl printBreak ; stdio.asm

.globl mostrar_tabla
; submenu1/mostrar_tabla.asm

.globl caracter_a_caracter2
; submenu2/caracter_a_caracter.asm
.globl palabra_a_palabra2
; submenu2/palabra_a_palabra.asm
.globl linea_a_linea2
; submenu2/linea_a_linea.asm

.globl caracter_a_caracter3
; submenu3/caracter_a_caracter.asm
.globl palabra_a_palabra3
; submenu3/palabra_a_palabra.asm
.globl linea_a_linea3
; submenu3/linea_a_linea.asm

menu0:
.ascii "\nPractica MORSE. v. 0.2.1\n"
.ascii "\n Menu DE OPCIONES:\n"
.ascii " 1) Ver tabla\n"
.ascii " 2) Texto a MORSE\n"
.ascii " 3) MORSE a Texto\n"
.ascii " S/s) Salir\n"
.asciz "\n >> Teclee una opcion: "

menu2:
.ascii "\n\nOpcion 2. Texto a MORSE\n"
.ascii " 1) Caracter a caracter\n"
.ascii " 2) Palabra a palabra\n"
.ascii " 3) Linea a linea\n"
.ascii " V/v) Volver\n"
.asciz "\n >> Elige: "

menu3:
.ascii "\n\nOpcion 3. MORSE a texto\n"
.ascii " 1) Caracter a caracter\n"
.ascii " 2) Palabra a palabra\n"
.ascii " 3) Linea a linea\n"
.ascii " V/v) Volver\n"
.asciz "\n >> Elige: "

incorrecta: .asciz "Opcion incorrecta!"

nucleo: ldx #menu0
jsr printStr
lda teclado

; comprueba que es un valor valido
cmpa #1
beq submenu1
cmpa #2
beq submenu2
cmpa #3
beq submenu3

anda #223 ; Convierte a mayusculas
cmpa #S
beq acabar

jsr printBreak
ldx #incorrecta
jsr printStr
bra nucleo

submenu1: jsr mostrar_tabla
bra nucleo

submenu2: ldx #menu2
; SALTO A LA SUBROUTINA DE submenu2
jsr printStr

; Debo pedir ahora la opcion del menu
lda teclado
jsr printBreak

cmpa #1
beq char_a_char2
cmpa #2
beq pal_a_pal2
cmpa #3
beq lin_a_lin2

anda #223 ; convierte a mayusculas
cmpa #V
beq nucleo

ldx #incorrecta
jsr printStr
beq submenu2
```

```
char_a_char2:
    jsr     caracter_a_caracter2
    bra     submenu2
```

```
pal_a_pal2:
    jsr     palabra_a_palabra2
    bra     submenu2
```

```
lin_a_lin2:
    jsr     linea_a_linea2
    bra     submenu2
```

```
submenu3:
    ldx     #menu3 ; SALTO A LA
DIRECCION DEL STRING DEL SUBMENÚ 3
    jsr     printStr
```

```
    ; Opcion del menu
    lda     teclado
    jsr     printBreak
```

```
    cmpa    #'1
    beq     char_a_char3
    cmpa    #'2
    beq     pal_a_pal3
    cmpa    #'3
    beq     lin_a_lin3
```

```
    anda    #223 ; convierte a mayusculas
```

```
    cmpa    #'V
    beq     nucleo
```

```
    ldx     #incorrecta
    jsr     printStr
```

```
    beq     submenu3
```

```
char_a_char3:
    jsr     caracter_a_caracter3
    bra     submenu3
```

```
pal_a_pal3:
    jsr     palabra_a_palabra3
    bra     submenu3
```

```
lin_a_lin3:
    jsr     linea_a_linea3
    bra     submenu3
```

```
    ; el programa acaba
acabar:
```

```
    jsr     printBreak
    clra
    sta     fin
```

```
    .area    FIJA (ABS)
    .org     0xFFFFE ; vector de RESET
    .word    nucleo
```

7. Ejemplos de pruebas

Se va a comprobar cada requisito del listado de ítems de evaluación, mencionando si se ha logrado o no ese apartado con imágenes de los resultados mostrados en terminal.

Ensambla? (comprobación nuestra)

```
(base) alumno@a4fc02:~/Descargas/COMPII_PRACTICAFINAL-main$ ./ensamblar.sh
ensambla > Opciones de depuracion desactivadas
ensambla > 13 modulos detectados
ensambla > Ensamblando... morse.asm
ensambla > Ensamblando... libraries/stdio.asm
ensambla > Ensamblando... libraries/stringH.asm
ensambla > Ensamblando... libraries/charMorse.asm
ensambla > Ensamblando... tables/chars_table.asm
ensambla > Ensamblando... tables/morse_table.asm
ensambla > Ensamblando... submenu1/mostrar_tabla.asm
ensambla > Ensamblando... submenu2/caracter_a_caracter.asm
ensambla > Ensamblando... submenu2/palabra_a_palabra.asm
ensambla > Ensamblando... submenu2/linea_a_linea.asm
ensambla > Ensamblando... submenu3/caracter_a_caracter.asm
ensambla > Ensamblando... submenu3/palabra_a_palabra.asm
ensambla > Ensamblando... submenu3/linea_a_linea.asm
ensambla > Enlazando/reuniendo... morse.rel libraries/stdio.rel libraries/stringH.rel libraries/charMorse.rel tables/chars_table.rel tables/morse_table.rel submenu1/mostrar_tabla.rel submenu2/caracter_a_caracter.rel submenu2/palabra_a_palabra.rel submenu2/linea_a_linea.rel submenu3/caracter_a_caracter.rel submenu3/palabra_a_palabra.rel submenu3/linea_a_linea.rel

ASlink >> -s
ASlink >> -m
ASlink >> -w
ASlink >> -u
ASlink >> morse.s19
ASlink >> morse.rel
ASlink >> libraries/stdio.rel
ASlink >> libraries/stringH.rel
ASlink >> libraries/charMorse.rel
ASlink >> tables/chars_table.rel
ASlink >> tables/morse_table.rel
ASlink >> submenu1/mostrar_tabla.rel
ASlink >> submenu2/caracter_a_caracter.rel
ASlink >> submenu2/palabra_a_palabra.rel
ASlink >> submenu2/linea_a_linea.rel
ASlink >> submenu3/caracter_a_caracter.rel
ASlink >> submenu3/palabra_a_palabra.rel
ASlink >> submenu3/linea_a_linea.rel
ensambla > Ejecutando...morse.s19
Reading symbols from 'morse.map'...

PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
1) Ver tabla
2) Texto a MORSE
3) MORSE a Texto
S/s) Salir
```

Implementar el menú principal y submenús.

```
PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
1) Ver tabla
2) Texto a MORSE
3) MORSE a Texto
S/s) Salir

>> Teclee una opcion: 2

OpciOn 2. Texto a MORSE
1) Caracter a caracter
2) Palabra a palabra
3) Linea a linea
V/v) Volver

>> Elige: v

PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
1) Ver tabla
2) Texto a MORSE
3) MORSE a Texto
S/s) Salir

>> Teclee una opcion: 3
```

```
OpciOn 3. MORSE a texto
1) Caracter a caracter
2) Palabra a palabra
3) Linea a linea
V/v) Volver

>> Elige: v

PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
1) Ver tabla
2) Texto a MORSE
3) MORSE a Texto
S/s) Salir
```

```
>> Teclee una opcion: s
morse.s19 : 28947 cycles, 60235 ms
(base) alumno@a4fc02:~/Descargas/COMPII_PRACTICAFINAL-main$
```

Opción 1 “Ver tabla”

```
Menu DE OPCIONES:
  1) Ver tabla
  2) Texto a MORSE
  3) MORSE a Texto
  S/s) Salir

>> Teclee una opcion: 1

1) TABLA MORSE:

A->.-
B->-...
C->-.-.
D->-..
E->.
F->...-
G->--.
H->....
I->..
J->.-.-
K->-.-
L->.-..
M->--
```

```
N->-..
O->---
P->.-.-
Q->-.-.-
R->.-.
S->...
T->-
U->...-
V->...-
W->.-.-
X->-...-
Y->-.-.-
Z->-.-..
0->-----
1->.-----
2->..---
3->...--
4->....-
5->.....
6->-.....
7->-.-...
8->-.-.-
9->-.-.-.
```

Opción 2 “Texto a MORSE”

```
PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
  1) Ver tabla
  2) Texto a MORSE
  3) MORSE a Texto
  S/s) Salir

>> Teclee una opcion: 2

OpciOn 2. Texto a MORSE
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  V/v) Volver

>> Elige: █
```

Opción 2.1 "Carácter a carácter" ✓

```
PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
  1) Ver tabla
  2) Texto a MORSE
  3) MORSE a Texto
  S/s) Salir

>> Teclee una opcion: 2

OpciOn 2. Texto a MORSE
  1) Carácter a carácter
  2) Palabra a palabra
  3) Línea a línea
  V/v) Volver

>> Elige: 1

2.1) Texto a MORSE (Carácter a Carácter):

P(.---.)
o(---)
N(-.)
n(-.)
O(---)
s(...)
( )
U(.---)
n(-.)
( )
1(.----)
0(----)
( )
P(.---.)
O(---)
R(.---)
F(.---)
A(-)
!
```

```
OpciOn 2. Texto a MORSE
  1) Carácter a carácter
  2) Palabra a palabra
  3) Línea a línea
  V/v) Volver

>> Elige: v

PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
  1) Ver tabla
  2) Texto a MORSE
  3) MORSE a Texto
  S/s) Salir

>> Teclee una opcion: s
morse.s19 : 20522 cycles, 29086 ms
(base) alumno@a4fc02:~/Descargas/COMPII_PRACTICAFINAL-main$
```

Opción 2.2 "Palabra a palabra" ✓

```
PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
  1) Ver tabla
  2) Texto a MORSE
  3) MORSE a Texto
  S/s) Salir

>> Teclee una opcion: 2

OpciOn 2. Texto a MORSE
  1) Carácter a carácter
  2) Palabra a palabra
  3) Línea a línea
  V/v) Volver

>> Elige: 2

2.2) Texto a MORSE (Palabra a Palabra):

Ey
. ---
ey
. ---
aqui
. --- . . .
ELRichMC
. --- . . . . . . . . . .
en
. ---
Minecraft
. --- . . . . . . . . . .
.
```

```
OpciOn 2. Texto a MORSE
  1) Carácter a carácter
  2) Palabra a palabra
  3) Línea a línea
  V/v) Volver

>> Elige: v

PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
  1) Ver tabla
  2) Texto a MORSE
  3) MORSE a Texto
  S/s) Salir

>> Teclee una opcion: s
morse.s19 : 25086 cycles, 21557 ms
(base) alumno@a4fc02:~/Descargas/COMPII_PRACTICAFINAL-main$
```

Opción 2.3 "Línea a línea"

```
PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
  1) Ver tabla
  2) Texto a MORSE
  3) MORSE a Texto
  S/s) Salir

>> Teclee una opcion: 2

OpciOn 2. Texto a MORSE
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  V/v) Volver

>> Elige: 3

2.3) Texto a MORSE (Linea a Linea):
Lorem ipsum dolor sit amet
... ..
Un vaso es un vaso
... ..
Cogito ergo sum
... ..
!
```

```
OpciOn 2. Texto a MORSE
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  V/v) Volver

>> Elige: v

PrActica MORSE. v. 0.2.1

Menu DE OPCIONES:
  1) Ver tabla
  2) Texto a MORSE
  3) MORSE a Texto
  S/s) Salir

>> Teclee una opcion: s
morse.sl9 : 40407 cycles, 96463 ms
(base) alumno@a4fc02:~/Descargas/COMPII_PRACTICAFINAL-main$
```


Opción 3 “MORSE a Texto” ✓

```
Practica MORSE. v. 0.2.1

Menu DE OPCIONES:
  1) Ver tabla
  2) Texto a MORSE
  3) MORSE a Texto
  S/s) Salir

>> Teclee una opcion: 3

Opcion 3. MORSE a texto
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  V/v) Volver

>> Elige: █
```

Opción 3.1 "Carácter a carácter" ✓

- Entrada correcta ✓
 - Encontrado, sacar carácter por pantalla, y volver a pedir ✓
 - No encontrado, mostrar error y volver al menú ✓

```
>> Elige: 1
3.1) MORSE a Texto (Caracter a Caracter):
... ->S
--- ->O
... ->S
.... CODIGO NO VALIDO

Opcion 3. MORSE a texto
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  V/v) Volver
```

- **Entrada incorrecta** ✓

- **Error longitud de código, mostrar error y volver al menú** ✓

```
>> Elige: 1
3.1) MORSE a Texto (Caracter a Caracter):
----- ERROR LONGITUD ENTRADA

Opcion 3. MORSE a texto
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  v/v) Volver

>> Elige: █
```

- **Espacio o enter con código vacío, mostrar error y volver al menú** ✓

```
>> Elige: 1
3.1) MORSE a Texto (Caracter a Caracter):

ERROR LONGITUD ENTRADA

Opcion 3. MORSE a texto
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  v/v) Volver

>> Elige: █
```

- **No es “-”, “.”. espacio, ni enter, mostrar error y volver a menú** ✓

```
>> Elige: 1
3.1) MORSE a Texto (Caracter a Caracter):
..5 ERROR CARACTER ENTRADA

Opcion 3. MORSE a texto
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  v/v) Volver

>> Elige: █
```

Opción 3.2 "Palabra a palabra" ✓

- Entrada correcta ✓
 - Finaliza palabra con dos espacios, muestra traducción y pide otra ✓

```
>> Elige: 2

3.2) MORSE a Texto (Palabra a Palabra):
... --- ...
SOS
... ..- .-.-. - .. -.-. .-
PRACTICA
... --- ...
SOS

Opcion 3. MORSE a texto
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  V/v) Volver

>> Elige: █
```

- Finaliza palabra con enter, muestra traducción y vuelve al menú (se muestra en la anterior imagen) ✓
- Se introduce código no válido, traduce lo leído, y vuelve al menú ✓

```
>> Elige: 2

3.2) MORSE a Texto (Palabra a Palabra):
... --- .... CODIGO NO VALIDO
SO

Opcion 3. MORSE a texto
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  V/v) Volver

>> Elige: █
```

- **Entrada incorrecta** ✓

- Error en longitud de código, traduce lo leído, y vuelve al menú ✓

```
>> Elige: 2

3.2) MORSE a Texto (Palabra a Palabra):
..... ----- ERROR LONGITUD ENTRADA
5

Opcion 3. MORSE a texto
    1) Caracter a caracter
    2) Palabra a palabra
    3) Linea a linea
    V/v) Volver

>> Elige: █
```

- No es “-”, “.”. espacio, ni enter, mostrar error y volver a menú ✓

```
>> Elige: 2

3.2) MORSE a Texto (Palabra a Palabra):
... --- ...5 ERROR CARACTER ENTRADA
S0

Opcion 3. MORSE a texto
    1) Caracter a caracter
    2) Palabra a palabra
    3) Linea a linea
    V/v) Volver

>> Elige: █
```

Opción 3.3 "Línea a línea" ✓

- Entrada correcta. ✓
 - Acepta varias palabras (2 espacios separador) y cuando se pulsa ENTER traduce y vuelve a pedir otra línea. ✓

```
>> Teclee una opcion: 3

Opcion 3. MORSE a texto
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  V/v) Volver

>> Elige: 3

3.3) MORSE a Texto (Linea a Linea):
... --- ... --- --- --- --- --- --- --- --- --- --- --- --- --- ---
PRACTICA COMPUTADORES II 2026
... --- ... --- --- --- --- --- --- --- --- --- --- --- --- --- ---
SOS SOS SOS
█
```

- Error de código no detectado, traduce lo leído y vuelve al menú. ✓

```
>> Elige: 3

3.3) MORSE a Texto (Linea a Linea):
... --- ... --- --- CODIGO NO VALIDO
SOS S

Opcion 3. MORSE a texto
  1) Caracter a caracter
  2) Palabra a palabra
  3) Linea a linea
  V/v) Volver

>> Elige: █
```

- Errores de longitud errónea, carácter erróneo, muestra cadena de error, traduce lo leído y vuelve al menú. ✓

```
>> Elige: 3
```

```
3.3) MORSE a Texto (Linea a Linea):
```

```
... --- ... --- ERROR LONGITUD ENTRADA  
SOS S
```

```
Opcion 3. MORSE a texto
```

```
1) Caracter a caracter
```

```
2) Palabra a palabra
```

```
3) Linea a linea
```

```
V/v) Volver
```

```
>> Elige: █
```

```
>> Elige: 3
```

```
3.3) MORSE a Texto (Linea a Linea):
```

```
... --- ... ---8 ERROR CARACTER ENTRADA  
SOS S
```

```
Opcion 3. MORSE a texto
```

```
1) Caracter a caracter
```

```
2) Palabra a palabra
```

```
3) Linea a linea
```

```
V/v) Volver
```

```
>> Elige: █
```

Evaluación:

A. (9 puntos): Funcionamiento del programa según las especificaciones anteriores.

Aparentemente, el programa cumple todas las especificaciones como se indica en anteriores páginas de este apartado. Los siguientes apartados de evaluación quedan a criterio del corrector, pero creemos conveniente destacar los siguientes aspectos:

B. (0,25 puntos): Uso de subrutinas y bibliotecas de subrutinas.

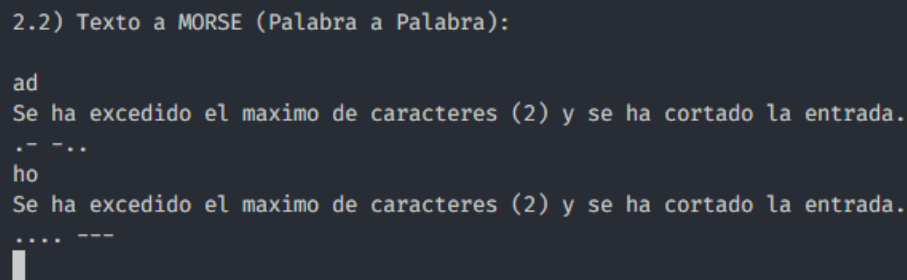
Uno de los aspectos más resaltables de la forma en la que se ha realizado el proyecto es el hecho de que tiene un alto nivel de compartimentalización, destacando el número de subrutinas. Se ha utilizado un centenar de saltos a subrutinas (96, para ser precisos), contando con un total de unas 18 funciones auxiliares principales y unas 30 funciones auxiliares secundarias.

Se ha utilizado una subrutina por cada una de las opciones de los menús: Morse a texto, carácter a carácter, mostrar tabla, texto a Morse, línea a línea... Gracias a este diseño tan estructurado se logra un proyecto más fácil de entender y mantener. Se han creado, asimismo, 3 bibliotecas de subrutinas, alojadas en la carpeta “*libraries*”:

- *charMorse.asm* contiene las funciones relativas a la traducción de texto a Morse y de Morse a texto, usando las correspondencias de los archivos *morse_table* y *chars_table*, respectivamente.
- *stdio.asm* contiene las funciones que permite abstraer el acceso directo a los puertos de entrada/salida de la máquina virtual, reutilizando código común y simplificando el desarrollo del resto de módulos.
- *stringH.asm* trata cadenas de caracteres. Su principal rutina es *strcmp*, utilizada para comparar dos cadenas, carácter a carácter, hasta encontrar una diferencia, o el terminador \0. Es fundamental para traducir de Morse a texto: comprueba si un código Morse introducido coincide con alguno de los de *morse_table*.

C. (0,50 puntos): Uso adecuado y responsable de la memoria.

GESTIÓN DE LA MEMORIA, APARTADOS 2.2 Y 2.3



```
2.2) Texto a MORSE (Palabra a Palabra):
ad
Se ha excedido el maximo de caracteres (2) y se ha cortado la entrada.
.- -..
ho
Se ha excedido el maximo de caracteres (2) y se ha cortado la entrada.
.... ---
```

Como la memoria no es ilimitada, hemos tenido que desarrollar un sistema de reserva de memoria para poder trabajar con la cadena de caracteres que se introduce desde teclado (si no reservásemos algunos bytes, se podrían llegar a sobrescribir bytes que contienen la información de nuestro propio programa, dando lugar a una vulnerabilidad llamada *buffer overflow*) [3].

Para ello, hemos reservado un total de 101 bytes, 1 de ellos para el \0 (aunque en el ejemplo se muestra con 2 por simplicidad) para almacenar esta cadena de entrada, para lo cual hemos usado la instrucción `.rmb` (LEVENTHAL, 1981).

```
cadena_introducida: .rmb    101
```

En esta cadena se almacenarán los caracteres que se vayan introduciendo hasta que se lleguen a introducir 100, momento en el cual se realizará la traducción a morse automáticamente y se pasará a la siguiente línea de entrada.

GESTIÓN DE LA MEMORIA, APARTADOS 3.2 Y 3.3

```
3.3) MORSE a Texto (Linea a Linea):  
.- .-  
Se ha excedido el maximo de caracteres (2) y se ha cortado la entrada.  
AA  
.-  
Se ha excedido el maximo de caracteres (2) y se ha cortado la entrada.  
A  
█
```

Utilizamos otra estrategia en el apartado 3 del menú. Como cada carácter Morse ocupa entre 1 y 5 caracteres punto y/o guion, es claramente más eficiente en memoria almacenar la traducción a caracteres alfanuméricos de los mismos:

```
traduccion: .rmb    101
```

Pues así podemos guardar entradas de hasta 100 códigos Morse, en lugar de sólo hasta 100 caracteres.

Para que funcione el sistema, igualmente se ha de guardar temporalmente el valor del código Morse que se introduce por teclado, lo cual se logra a través de esta variable:

```
cadena_introducida: .rmb    6
```

La cual se va sobrescribiendo cada vez que se introduce un carácter nuevo.

Destacamos otra ventaja de almacenar la traducción en lugar de los propios códigos Morse introducidos: como lo que se almacenan son las traducciones, no existe la posibilidad de que la entrada se corte con un código incompleto porque este supere el centésimo carácter (lo cual devolvería un error). Con este sistema, la entrada se corta una vez terminado de escribir el centésimo código Morse.

El funcionamiento de este sistema es el mismo que el del apartado 2: si se introducen por teclado 100 caracteres Morse, la entrada se corta y se traduce automáticamente, pasando a pedir la siguiente línea por teclado inmediatamente después.

D. (0,25 puntos): Claridad y calidad del código fuente.

Gracias a la intensiva modularización del proyecto, la clara documentación de las bibliotecas de subrutinas y la abundancia de comentarios en todos los archivos, se logra una mayor claridad en el proyecto que facilitará su modificación en caso de que sea necesario.

8. Conclusiones, referencias y bibliografía

Conclusiones

El haber tenido la oportunidad de realizar esta práctica ha supuesto para los estudiantes la adquisición de las competencias incluidas en la guía docente de la asignatura de Computadores 2, sobre todo la general CC9, la específica IC1 y las transversales CT5 y CT6:

- «CC9.- Capacidad de conocer, comprender y evaluar la estructura y arquitectura de los computadores, así como los componentes básicos que los conforman».
- «IC1. - Capacidad de diseñar y construir sistemas digitales, incluyendo computadores, sistemas basados en microprocesador y sistemas de comunicaciones».
- «CT5. - Comunicación oral y escrita».
- «CT16. - Capacidad de aplicar los conocimientos en la práctica».

Añadidamente, creemos que esta práctica ha supuesto la mejora en nuestras habilidades técnicas de:

- Programación, estructuración de archivos y proyectos, análisis de tablas y mapas de datos, ofimática, manejo de otros sistemas operativos (como Linux), manejo de la terminal de comandos del propio sistema...

Y hemos observado también notablemente una mejora en las siguientes habilidades blandas, tales como trabajo en equipo, detección de fallos, resolución eficiente de problemas y comunicación y expresión oral (la defensa) como escrita (este informe).

Referencias

[1]: NASA/JPL-CALTECH. Reusado en: Popular Science. (2024). *What is on the Voyager Golden Record?* (<https://www.popsci.com/science/what-is-on-voyager-golden-record/>)

[2]: BLANCO DE LA IGLESIA, D. [UGULBERTO], RODRÍGUEZ FERNÁNDEZ, N. [NICO-RF]. (7 de mayo de 2026). *Repositorio de GitHub "Ugulberto/COMPII_PRACTICAFINAL"*. GitHub https://github.com/Ugulberto/COMPII_PRACTICAFINAL.

[3]: SHIREY, R. (Agosto de 2007). *Internet Security Glossary, Version 2*. Network Working Group. <https://www.rfc-editor.org/rfc/rfc4949> doi:10.17487/RFC4949.

(Obtenido de en.wikipedia.org/wiki/Buffer_overflow#Technical_description)

Bibliografía y webgrafía

AKUMAS, C. (s.f.) *Cheatsheet Collection: 6502, 6280, 6309, 6809* ChibiAkumas. <https://www.chibiakumas.com/book/CheatSheetCollection.pdf>.

AKUMAS, C. (s.f.) *Learn Multi platform 6809 Assembly Programming... 8 bit resurrection!* ChibiAkumas. <https://www.chibiakumas.com/6809/>.

AUTOEDICIÓN. (1979). *6809 Assembler*. Computerware™ Software Services. https://deramp.com/swtpc.com/SSB_DOC/Computerware_6809_Assembler.pdf

GONZÁLEZ TALAVÁN, G., DEPARTAMENTO DE INFORMÁTICA Y AUTOMÁTICA DE LA UNIVERSIDAD DE SALAMANCA. (2026). *Computadores II. 1º Grado en Ingeniería Informática*. (Apuntes de la asignatura).

GONZÁLEZ TALAVÁN, G., DEPARTAMENTO DE INFORMÁTICA Y AUTOMÁTICA DE LA UNIVERSIDAD DE SALAMANCA. (2025, 2010-2019). *Página oficial de Computadores II. Prácticas de la 1 a la 10*. AVELLANO EN GGTSOFT. avellano.usal.es/compji.

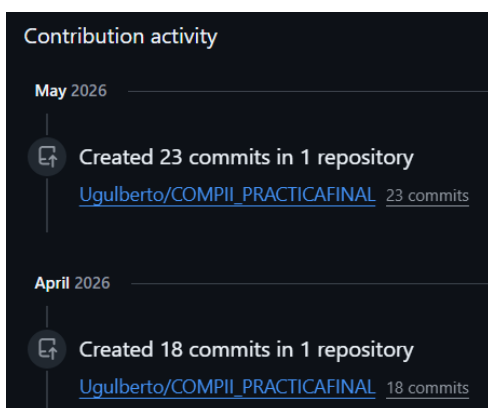
LEVENTHAL, L. A. (1981). *Assembly Language Programming: 6809*. Osborne Publishing. Revisado en *The Internet Archive*, [<https://archive.org/>]. archive.org/details/6809assemblylang0000leve.

TRIBKI, T. (14 de enero de 2019). *Motorola 6809 Cheatsheet*. Revisado en *GitHub*, [<https://github.com/>]. <https://github.com/Ta0uf19/Motorola-6809-Cheatsheet>.

Anexo I. Historial de *commits* de GitHub

Muchos de los *commits* (confirmaciones) realmente realizados (como se puede comprobar al final de la página) proceden de *branches* (ramas) que están fusionadas, por lo que no aparecen en el log que se genera al ejecutar el comando *git log*.

FECHA	Autor	Nombre del commit	SHA key
May 7 20:36:00	David	Update ensamblar.sh	6a15fb8
May 7 15:41:16	David	cambio de formato en chars_table y ajuste de límite de caracteres en submenu 2	8306127
May 6 23:35:28	David	manejo eficiente de memoria y subrutinas para menos lineas	c4a88f6
May 6 18:08:29	David	submenu3 y Estructura de archivos organizada y corregida	bb6e70c
May 6 10:12:43	Nico	versión 0.3.2 corregida	f274899
May 3 18:33:09	Nico	act mia	37b1078
Apr 26 13:52:39	David	fix opcion 1	94d86df
Apr 21 17:49:12	Nico	minor changes	376327b
Apr 21 17:47:50	Nico	COMENTARIO MOSTRAR_TABLA	75494c3
Apr 21 17:46:32	Nico	delete old alphabet	12671b1
Apr 21 17:44:25	Nico	cajitas de comentarios	75b63ab
Apr 21 17:39:46	David	Refactor Morse code table handling	5d5ba9f
Apr 21 17:39:32	David	Add mostrar_tabla.asm to display Morse table	08229a1
Apr 21 17:19:10	David	Refactor stdio.asm for consistency and clarity	3743b34
Apr 21 17:18:29	David	Refactor morse.asm to include Morse code table	bcdc429
Apr 21 16:46:04	David	Create morse_table.inc	d5e48f5
Apr 21 16:45:48	David	Add Morse code table in morse_table.asm	f73b72c
Apr 21 16:45:07	David	Refactor morse.asm for improved structure and clarity	a22c066
Apr 21 12:35:04	Nico	minor changes 1	1f1a015
Apr 20 13:06:58	Nico	Coherencia+functs	fb629f9
Apr 19 18:12:20	Nico	TT y pendientes	c049826
Apr 19 18:07:51	Nico	inicial	be93be4
Apr 19 17:42:08	Nico	Semicolons in stringH.asm	1052901
Apr 19 17:37:11	Nico	Semicolons in stdio.asm	5c976f6
Apr 19 12:18:09	David	inicial	3247e59



Historial de Nicolás



Historial de David
